1 ALTA DE PERSONAL

```
--Aixi es dona d'alta a un metge--
-- 1) Inserim a PERSONAL--
INSERT INTO PERSONAL (id_personal, nom, cognoms, dni, adreca, telefon)
VALUES (101, 'Gabriela', 'Borrull López', '46192775', 'C/ Girona 25',
'640137640');

-- 2) Inserim a METGE--
INSERT INTO METGE (id_personal, especialitat_metge,
dades_curriculum_metge)
VALUES (101, 'Cardiologia', '10 anys d''experiència en cardiologia
clínica');

--Aixi es dona d'alta a un infermer/a--
INSERT INTO PERSONAL (id_personal, nom, cognoms, dni, adreca, telefon)
VALUES (102, 'Jose', 'Da Silva', '87654321B', 'Av. Catalunya 55',
'611222333');

INSERT INTO INFERMERIA (id_personal, especialitat_infermeria,
dades_curriculum_infermera, id_metge)
VALUES (102, 'Infermeria quirúrgica', 'Experiència en UCI', 101);  --
depèn del metge 101

--vari--
INSERT INTO PERSONAL (id_personal, nom, cognoms, dni, adreca, telefon)
VALUES (103, 'Pau', 'Fernandez López', '11223344C', 'C/ Massanes 13',
'699888777');

INSERT INTO VARI (id_personal, tipus_tasca)
VALUES (103, 'Neteja');
```

Primer faig INSERT(sempre va a PERSONAL) perquè totes les persones del centre són personal.
Segon INSERT(depèn del rol)

- Si és metge: METGE
- Si és infermer/a: INFERMERIA
- Si és neteja vari: VARI

id_personal és la clau que connecta PERSONAL amb la taula específica i **id_metge** a INFERMERIA: permet saber si depèn d'un metge.

manteniment.alta_personal

```
import tkinter as tk
from tkinter import ttk, messagebox
import psycopg2
```

```
from manteniment.alta_pacient import validar_dni_nie

def registrar_personal(id_personal, nom, cognoms, dni, adreca, telefon,
rol, especialitat, curriculum, id_metge):
    try:
        conn=psycopg2.connect(
            host="192.168.56.112",
            database="HospitalHarmoniaBlanes",
            user="postgres",
            password="admin"
        )
        cur=conn.cursor()

            #segons personal
        cur.execute("""
                INSERT INTO PERSONAL (id_personal, nom, cognoms, dni,
adreca, telefon)
                VALUES (%s, %s, %s, %s, %s, %s)
            """, (id_personal, nom, cognoms, dni, adreca, telefon))

            #segons rol
        if rol == "Metge":
                cur.execute("""
                    INSERT INTO METGE (id_personal, especialitat_metge,
dades_curriculum_metge)
                    VALUES (%s, %s, %s)
                """, (id_personal, especialitat, curriculum))

        elif rol == "Infermer/a":
                cur.execute("""
                    INSERT INTO INFERMERIA (id_personal,
especialitat_infermeria, dades_curriculum_infermera, id_metge)
                    VALUES (%s, %s, %s, %s)
                """, (id_personal, especialitat, curriculum, id_metge))

        elif rol == "Personal de neteja":
                cur.execute("""
                    INSERT INTO VARI (id_personal, tipus_tasca)
                    VALUES (%s, %s)
                """, (id_personal, "Neteja"))

        conn.commit()
```

```
        cur.close()
        conn.close()

        messagebox.showinfo("Correcte", "Personal registrat
correctament")

    except Exception as e:
        messagebox.showerror("Error", str(e))
def finestra_alta_personal():
    root = tk.Toplevel()
    root.title("Alta de Personal (ADMIN)")
    root.geometry("400x600")

    # ID
    tk.Label(root, text="ID Personal").pack()
    entry_id = tk.Entry(root)
    entry_id.pack()

    # Nom
    tk.Label(root, text="Nom").pack()
    entry_nom = tk.Entry(root)
    entry_nom.pack()

    # Cognoms
    tk.Label(root, text="Cognoms").pack()
    entry_cognoms = tk.Entry(root)
    entry_cognoms.pack()

    # DNI
    tk.Label(root, text="DNI").pack()
    entry_dni = tk.Entry(root)
    entry_dni.pack()

    # Adreça
    tk.Label(root, text="Adreça").pack()
    entry_adreca = tk.Entry(root)
    entry_adreca.pack()

    # Telèfon
    tk.Label(root, text="Telèfon").pack()
    entry_telefon = tk.Entry(root)
    entry_telefon.pack()
```

```
    # Rol
    tk.Label(root, text="Rol").pack()
    combo_rol = ttk.Combobox(root, values=["Metge", "Infermer/a",
"Personal de neteja"])
    combo_rol.pack()

    # Especialitat
    tk.Label(root, text="Especialitat (si aplica)").pack()
    entry_especialitat = tk.Entry(root)
    entry_especialitat.pack()

    # Curriculum
    tk.Label(root, text="Curriculum (si aplica)").pack()
    entry_curriculum = tk.Entry(root)
    entry_curriculum.pack()

    # ID Metge (només per infermeria)
    tk.Label(root, text="ID Metge responsable (només
infermeria)").pack()
    entry_id_metge = tk.Entry(root)
    entry_id_metge.pack()

    # BOTÓ GUARDAR
    def guardar():
        try:
            id_personal = int(entry_id.get())
        except:
            messagebox.showerror("Error", "ID Personal ha de ser un
número")
            return

        nom = entry_nom.get()
        cognoms = entry_cognoms.get()
        dni = entry_dni.get()
        adreca = entry_adreca.get()
        telefon = entry_telefon.get()
        rol = combo_rol.get()
        especialitat = entry_especialitat.get()
        curriculum = entry_curriculum.get()
        id_metge = entry_id_metge.get() if rol == "Infermer/a" else
None

        if not validar_dni_nie(dni):
```

```
            messagebox.showerror("Error", "DNI o NIE no vàlid")
            return

        registrar_personal(id_personal, nom, cognoms, dni, adreca,
telefon,
                           rol, especialitat, curriculum, id_metge)

        root.destroy()

    tk.Button(root, text="Guardar", command=guardar).pack(pady=15)

    root.mainloop()
```

pantalles.menu_admin (actualitzat)

```
import tkinter as tk
from login.gestio_usuaris import tornar_login

from manteniment.alta_personal import finestra_alta_personal

def abrir_admin(con):

    win = tk.Tk()
    win.title("Panel Admin")
    win.geometry("400x350")
    cur = con.cursor()

    # proba real de permisos


    tk.Label(win, text="MENU ADMIN", font=("Arial", 18)).pack(pady=20)

    tk.Button(win, text="Crear usuaris", width=25).pack(pady=5)
    tk.Button(win, text="Desbloquejar comptes", width=25).pack(pady=5)
    tk.Button(win, text="Veure logs", width=25).pack(pady=5)
    tk.Button(win, text="Alta de personal",
command=finestra_alta_personal).pack()

    tk.Button(win, text="Sortir", command=win.destroy).pack(pady=20)
    tk.Button(win, text="Tornar al login", command=lambda:
tornar_login(win)).pack(pady=20)

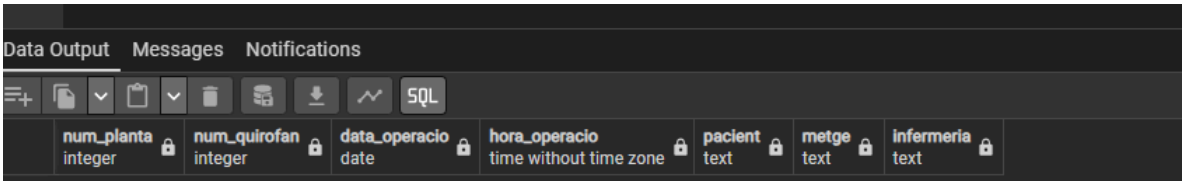
    win.mainloop()
```

4 CONSULTA D'OPERACIONS PER DIA I QUIRÒFAN

```
SELECT
    q.num_planta,
    q.num_quirofan,
    o.data_operacio,
    o.hora_operacio,
    p.nom_pacient || ' ' || p.cognoms_pacient AS pacient,
    per_m.nom || ' ' || per_m.cognoms AS metge,
    STRING_AGG(per_i.nom || ' ' || per_i.cognoms, ', ') AS infermeria
FROM OPERACIO o
INNER JOIN QUIROFAN q ON o.id_quirofan = q.id_quirofan
INNER JOIN PACIENT p ON o.id_pacient = p.id_pacient
INNER JOIN METGE m ON o.id_metge = m.id_personal
INNER JOIN PERSONAL per_m ON m.id_personal = per_m.id_personal
LEFT JOIN ASSISTENCIA_OPERACIO ao ON o.id_operacio = ao.id_operacio
LEFT JOIN INFERMERIA inf ON ao.id_infermeria = inf.id_personal
LEFT JOIN PERSONAL per_i ON inf.id_personal = per_i.id_personal
WHERE o.data_operacio = DATE '2026-05-01'
GROUP BY q.num_planta, q.num_quirofan, o.data_operacio,
o.hora_operacio, pacient, metge
ORDER BY q.num_planta, q.num_quirofan, o.hora_operacio;
```

q.num_planta, q.num_quirofan: es el quiròfan real.
INNER JOIN METGE + INNER JOIN PERSONAL: perqueè METGE només té especialitat, el nom està a PERSONAL.
LEFT JOIN ASSISTÈNCIA_OPERACIÓ: pot haver-hi 0 o més infermers.
LEFT JOIN INFERMERIA + PERSONAL: per obtenir el nom els infermers.
STRING_AGG: lo que fa es juntar tots els infermers en una sola línea.
WHERE data_operacio=...: per filtrar pel dia.
GROUP BY: finalment per que es necessari per STRING_AGG.

Execució:



6 TRIGGER PER EVITAR SOLAPAMENT D'OPERACIONS

El **solapament** amb el **nostre model** hi han dues operacions que **NO poden tenir:** mateix id_quirofan, mateix data_operacio i mateix hora_operacio.

```
--Funció del trigger--
```

```sql
CREATE OR REPLACE FUNCTION evitar_solapament_operacions()
RETURNS TRIGGER AS $$
DECLARE
    conflictes INT;
BEGIN
    SELECT COUNT(*)
    INTO conflictes
    FROM OPERACIO
    WHERE id_quirofan = NEW.id_quirofan
      AND data_operacio = NEW.data_operacio
      AND hora_operacio = NEW.hora_operacio
      AND id_operacio <> COALESCE(NEW.id_operacio, -1);

    IF conflictes > 0 THEN
        RAISE EXCEPTION 'Ja existeix una operació en aquest quiròfan a
aquesta data i hora.';
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

--Trigger associat--
CREATE TRIGGER trg_evitar_solapament_operacions
BEFORE INSERT OR UPDATE ON OPERACIO
FOR EACH ROW
EXECUTE FUNCTION evitar_solapament_operacions();
```

NEW.camp: és la fila que s'està intentant inserir o modificar.
COUNT(9*: NEW.id_operacio: evita comptar-se a si mateixa en un UPDATE.
RAISE EXCEPTION: per atura l'INSERT i el UPDATE i mostra error.
BEFORE INSERT OR UPDATE: s'executara abans de guardar.

6 FUNCIÓ PER SABER LES VISITES D'UN DIA
Les taules implicades serias: VISITA, PACIENT i METGE+PERSONAL

```sql
--FUNCIÓ PER SABER LES VISITES D'UN DIA--
--Funcio PL/pgSQL--
CREATE OR REPLACE FUNCTION visites_dia(p_data DATE)
RETURNS TABLE (
    hora TIME,
    pacient TEXT,
    metge TEXT,
    diagnostic TEXT
```

```
) AS $$
BEGIN
    RETURN QUERY
    SELECT
        v.hora,
        p.nom_pacient || ' ' || p.cognoms_pacient AS pacient,
        per_m.nom || ' ' || per_m.cognoms AS metge,
        v.diagnostic
    FROM VISITA v
    JOIN PACIENT p ON v.id_pacient = p.id_pacient
    JOIN METGE m ON v.id_metge = m.id_personal
    JOIN PERSONAL per_m ON m.id_personal = per_m.id_personal
    WHERE v.data = p_data
    ORDER BY v.hora;
END;
$$ LANGUAGE plpgsql;

--per cridar la funcio:--
SELECT * FROM visites_dia('2026-05-01');
```

RETURNS TABLE: la funció retorna una taula.
RETURN QUERY: el resultat de la SELECT és el retorn.
INNER JOIN METGE + PERSONAL: per obtenir el nom del metge.
ORDER BY v.hora: per ordenar les visites del dia.

Execució:

